Memory Residuals: An $O(1)$-Storage, $O(1)$-Routing Recurrent Memory Primitive for Frozen Pretrained Language Models

Anonymous Authors
Anonymous Affiliation
anonymous@example.com

Abstract

We introduce **Memory Residuals**, a fixed-size jointly-trained recurrent memory matrix $M_c \in \mathbb{R}^{K \times d}$ that an *otherwise frozen* pretrained LLM reads and writes through a depth-routed cross-attention residual. The construction occupies a previously empty corner of the long-horizon-memory design space: $\mathcal{O}(1)$ storage in chain length, $\mathcal{O}(1)$ per-token routing cost, and no backbone fine-tuning. Prior recurrent-memory recipes (Block-Recurrent, RMT, AutoCompressors) hit the constant-cost target but unfreeze the LM; retrieval and Memorising Transformers keep the backbone frozen but pay $\mathcal{O}(T)$ in storage or context. The primitive decomposes into four loosely-coupled segments—a Perceiver-style writer, a per-token readout, a depth router, and a single-scalar memory-channel floor—and the segmentation is itself a contribution: each piece can be replaced independently. Empirically, with $\sim$6% trainable-parameter overhead on a frozen Qwen3-0.6B and $\sim$1.5 h of training on a single H100, the primitive lowers callback cross-entropy on LongMemEval-S by $+1.32 \pm 0.53$ nats (four seeds, 95 % bootstrap CI $[+1.11, +1.55]$), preserves at 1.7B ($+0.978 \pm 0.091$ nats, n=3 seeds, every seed positive), and transfers without retraining to two unseen callback corpora at 1.7B ($\sim$70% of the in-domain gain). A matched control suite (chain-shuffle, evidence-redaction, oracle-RAG) localises the gain to *chain-conditional context compression* rather than per-fact recall, and three single-flag interventions (§7)—a per-category breakdown, a sparse-writer ablation on a random-alphanumeric-code corpus ($+1.33$ nats with positive chain-shuffle), and a contrastive auxiliary (Δ_{sh}:$0 \to +0.010$ without disturbing the headline)—resolve the "isn't this just a learned chain prior?" critique affirmatively. We treat the numbers as proof-of-life and the segmentation as the durable contribution; the deployment target is a life-long conversational agent on a frozen LLM whose memory module is trainable in hours, not weeks, and swappable rather than re-fine-tuned.

1 Introduction

The asymptotic design space of long-horizon LLM memory. The operating cost of a long-horizon memory mechanism over a chain of T sessions breaks into three axes: *storage* (state size as a function of T), *routing* (per-token inference cost paid to consult that state), and whether the LM *backbone is frozen*. Existing approaches sit in distinct corners of this cube (Table 1): long-context attention pays $\mathcal{O}(T^2)$ routing; RAG keeps the backbone frozen but the index is $\mathcal{O}(T)$ and per-query inference adds $\mathcal{O}(kS)$ context tokens; the recurrent-memory family (Transformer-XL [4], RMT [2], Block-Recurrent [8], AutoCompressors [3], Compressive Transformer [20]) achieves $\mathcal{O}(K)$ storage and routing but every prior recipe unfreezes the LM during training. Memorising Transformers [27] keep

Preprint. Under review.

the backbone frozen but pay $\mathcal{O}(T)$ in storage. *No prior recipe sits in the $\mathcal{O}(1)$/$\mathcal{O}(1)$/frozen-backbone corner with a memory module that learns useful long-horizon behaviour.* Memory Residuals is, to our knowledge, the first construction in that corner.

Table 1: Asymptotic cost of long-horizon memory mechanisms in chain length T. *Storage*: state size as a function of T. *Routing*: per-token LM inference cost added by consulting the memory. *Backbone*: whether the recipe trains with the LM weights frozen. K denotes a method-specific fixed memory budget; k the retrieval top-k; S the per-session length. Memory Residuals occupies the previously-empty $\mathcal{O}(1)$ / $\mathcal{O}(1)$ / frozen corner.

mechanism	storage in T	routing per token	frozen backbone
long context (RoPE-extended)	$\mathcal{O}(T)$ ctx tokens	$\mathcal{O}(T)$	yes
RAG [12]	$\mathcal{O}(T)$ index	$\mathcal{O}(\log T) + \mathcal{O}(kS)$ ctx	yes
Memorising Tx [27]	$\mathcal{O}(T)$ kNN store	$\mathcal{O}(\log T) + \mathcal{O}(k)$	no
Transformer-XL [4], Compressive [20]	$\mathcal{O}(K)$ KV cache	$\mathcal{O}(K)$	no
RMT [2], Block-Rec [8], AutoComp [3]	$\mathcal{O}(K)$ tokens	$\mathcal{O}(K)$	no
Memory Residuals (ours)	$\mathcal{O}(\mathbf{K})$ matrix	$\mathcal{O}(\mathbf{K})$	**yes**

This corner is the only point in the design space simultaneously cheap to deploy atop an existing pretrained LLM, cheap to run at inference, and bounded in state regardless of conversation length. A recipe that learns useful behaviour here is the natural target for a life-long conversational agent on a fixed backbone, where retraining the LM is prohibitive but constant per-conversation cost is required.

What this paper contributes. (1) An $\mathcal{O}(1)$/$\mathcal{O}(1)$/frozen-backbone recurrent memory primitive: a fixed-size jointly-trained matrix $M_c \in \mathbb{R}^{K \times d}$ (K=128) read and written through a depth-routed cross-attention residual, trained end-to-end with LM-NLL on chain-callback supervision; the LM backbone is never updated. (2) **Composable segments** designed for follow-on work: a Perceiver-style *writer* (§3.1), per-token *readout* (§3.2), *depth router* (§3.3), and a single-scalar α-*floor* (the only auxiliary loss); interfaces are narrow and each piece can be swapped independently. (3) A **chain-specificity measurement framework**: matched controls (chain-shuffle Δ_{sh}, evidence-redaction $\mathrm{evidence_lift}$, oracle-RAG) that separate "chain-conditional context compression" from "literal evidence retrieval". (4) Empirical proof-of-life: $+1.32 \pm 0.53$ nats at 0.6B (four seeds, 95 % bootstrap CI $[+1.11, +1.55]$) and $+0.978 \pm 0.091$ nats at 1.7B (n=3 seeds, every seed positive); transfer to two unseen callback corpora at 1.7B recovers $\sim$70% of the in-domain gain. The $\Delta_{\mathrm{sh}} \approx 0$ "learned chain prior" critique yields to three single-flag interventions (§7): a per-category breakdown surfaces hidden chain-conditional positives in four of six LongMemEval categories; a sparse-writer ablation produces $+1.33$ nats with positive chain-shuffle on a random-code corpus; a contrastive auxiliary loss lifts chain-shuffle from ≈ 0 to $+0.010$ without disturbing the headline.

2 Background and Related Work

Long-context Transformers. Recent benchmarks (NoLiMa [18], RULER [7], Lost-in-the-Middle [13]) show that even when the context window is nominally long, effective use of distant positions degrades sharply. We sidestep context extension entirely, keeping the window at the 4 k pretraining default and routing past-session information through a fixed-size memory matrix.

Retrieval-augmented generation. RAG [12] indexes past sessions and appends retrieved sessions to the LM context. The closest competitive simple-pipeline RAG with long-context LMs is DOS-RAG [11]; our RAG baselines mirror that recipe (per-session BM25 and MiniLM-L6 dense embeddings, no re-ranking). An oracle cell gives the model gold evidence-session indices directly, as a strict upper bound on simple-pipeline RAG.

Recurrent / compressive memory in pretrained LLMs. Transformer-XL [4] and Compressive Transformer [20] introduced segment-level and compressed-tier recurrence. RMT [2] and Block-Recurrent Transformer [8] thread a prepended memory token through segments. Memorising Transformers [27] use kNN over past keys; AutoCompressors [3] compresses prefixes into soft tokens. All five live in the $\mathcal{O}(K)$/$\mathcal{O}(K)$ column of Table 1 but unfreeze the LM during training; Memorising Transformers keeps the backbone frozen at inference but pays $\mathcal{O}(T)$ in storage. The

$\mathcal{O}(K)$/$\mathcal{O}(K)$/frozen-backbone combination is missing. Two architectural choices put memres there: the memory is read off the sequence axis through a depth-wise attention residual [10] (zero context-length cost), and the backbone is frozen during training (any callback improvement must flow through the memory module—leak-controlled by construction).

Attention residuals. Our depth-router builds on Attention Residuals [10], which replaces uniform depth-wise addition with a softmax over intermediate sources. We register the per-position memory readout m^t as a foundational source in that pool.

The RAG-vs-recurrent-memory comparison. LongMemEval [26] treats retrieval as the standard baseline; we extend that comparison by using a *frozen* backbone in both arms, running an *oracle* RAG cell with gold evidence positions, and reporting chain-shuffle and evidence-redacted controls for both methods.

3 Memory Residuals

The primitive has three components: a recurrent *writer* that updates an internal state $M_c \in \mathbb{R}^{K\times d}$ per session (§3.1); a per-token *readout* that produces a residual stream contribution $m^t \in \mathbb{R}^{S\times d}$ (§3.2); and a *depth router* that chooses how strongly each layer reads from the memory pathway (§3.3). All three are jointly trained end-to-end with LM-NLL on chain-callback supervision; the only auxiliary loss is a soft floor on the depth-router's memory channel.

3.1 Writer: two-stage QKV competition with iterative extraction

Let $M_c^{t-1} \in \mathbb{R}^{K\times d}$ be the persistent latent state prior to session t and $C_t \in \mathbb{R}^{N_t\times d}$ the token embeddings of the just-completed session. **Stage 1 (extraction).** A learnable query bank $M_{\text{in}} \in \mathbb{R}^{K\times d}$ pulls salient features from C_t via a Perceiver-style [9] L_E-step refinement: $E^{(0)} = \text{CrossAttn}(M_{\text{in}}, C_t)$ and $E^{(\ell)} = E^{(\ell-1)} + \text{CrossAttn}(E^{(\ell-1)}, C_t)$ for $\ell \in \{1,\dots,L_E\}$ (L_E=4, K=128 throughout). **Stage 2 (judge, slot-axis competition).** We concatenate $P_{\text{judge}} = [M_c^{t-1} \,\|\, E^{(L_E)}] \in \mathbb{R}^{2K\times d}$ and apply a slot-attention [14] update with QK-LayerNorm [29, 1]:

$$A = \underset{k\in[K]}{\text{Softmax}}\left(\frac{\widehat{Q}\,\widehat{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{K\times 2K}, \quad M_c^t = \text{LN}\big(M_c^{t-1} + \text{GRUCell}(M_c^{t-1}, A\,P_{\text{judge}}W_V)\big)\,. \tag{1}$$

The slot-axis softmax forces the K persistent slots to compete to *tile* the $2K$ candidates into disjoint specialist subsets, removing the symmetric uniform fixed point a candidate-axis softmax would admit.

3.2 Readout: per-token cross-attention into M_c

For session $X \in \mathbb{R}^{S\times d}$, the readout is a cross-attention with token embeddings as queries and M_c^t as keys/values, refined through a D=4-step Perceiver-style stack to produce $m^t \in \mathbb{R}^{S\times d}$. The readout depth is one of two load-bearing design choices the ablation (§9) identifies.

3.3 Depth router with α-floor

We register m^t as the foundational source b_{-1} in the attention-residual pool of [10], alongside per-block delta sources $b_0,\dots,b_{n-1}$. Each per-block routing sublayer computes a softmax over the pool with a *memory-channel floor* auxiliary loss:

$$\mathcal{L}_{\text{floor}} = \lambda_{\text{floor}} \cdot \max(0, \alpha^*_{\text{floor}} - \mathbb{E}\,\alpha_{\text{mem}})^2, \tag{2}$$

with $\lambda_{\text{floor}} = 0.5$, target $\alpha^*_{\text{floor}} = 0.10$. The router is initialised so the residual stream matches the bare backbone at step 0 (soft ± 4 bias chosen for non-saturated softmax gradients from step 1).

3.4 Frozen-backbone training and segment composability

The backbone—Qwen3-0.6B or Qwen3-1.7B [19]—is frozen (`lr_backbone`=0). Only memres parameters train: 41.5 M (∼6 % overhead) at 0.6B and 164.8 M (∼8 % overhead) at 1.7B, for 1000

(0.6B) / 1500 (1.7B) TBPTT steps on the LongMemEval-S 450-chain training split. The objective is plain LM-NLL on callback tokens plus Eq. 2.

The four components above are deliberately *loosely coupled*: each is small, single-purpose, and replaceable without touching the others. The segments interact through two narrow interfaces: $M_c \in \mathbb{R}^{K \times d}$ between writer and readout, and a per-layer scalar mix $\alpha_l \in [0, 1]$ between router and backbone. Swapping the writer (e.g. for a state-space model) leaves readout, router, and floor unchanged; swapping the depth-router for a learned per-layer gate leaves M_c unchanged. All four costs are T*-independent* (Table 2), placing the construction in the $\mathcal{O}(1)/\mathcal{O}(1)$ corner of Table 1. Most prior recurrent-memory recipes bundle writer, readout, and gate into a monolithic LM modification; we ship the segmentation as a contribution so each piece has a single testable job and follow-on work can replace it. The ablations in §9 verify each segment is individually load-bearing.

Table 2: Memres segments, I/O shapes, and per-session/token cost. T chain length, S session length, K=128 memory width, L backbone layers, L_E=D=4 writer/readout depths. All four are independent of T.

segment	I/O shapes	cost per session / token	natural swap axis
Writer (§3.1)	$(M_c^{t-1}, C_t) \to M_c^t$	$\mathcal{O}(L_E K S d)$ / session	extractor family (Perceiver→Mamba…)
Readout (§3.2)	$(M_c^t, h^t) \to m^t$	$\mathcal{O}(D K d)$ / token	query family, depth D
Depth router (§3.3)	layer sources → mix	$\mathcal{O}(L \cdot \lvert\text{sources}\rvert)$ / token	router parametrisation
α-floor (Eq. 2)	scalar aux loss	$\mathcal{O}(1)$ / step	anti-collapse prior

4 RAG Baselines and Eval Harness

RAG cells. We use a simple-pipeline RAG comparable to DOS-RAG [11], indexing each LongMemEval-S session as a document. Four cells per backbone: BM25 TOP-1 (`rank_bm25`), BM25 TOP-3, DENSE TOP-3 (`all-MiniLM-L6-v2`), and ORACLE TOP-3 (gold `chain_evidence_positions`). Retrieved sessions are concatenated into the LM's context window and callback-token CE is scored under the bare pretrained Qwen3 backbone. The oracle cell is a strict upper bound on the simple pipeline.

Eval harness. All numbers come from the same callback-aware evaluator (`tools/eval_callback.py`) on the same 50-chain corpus (`lme_val_s512_evpos.pt`). CE is computed only over *callback answer tokens* (1.4 % of total chain tokens), avoiding the $\sim$70$\times$ dilution from token-averaging. For every cell we report: Δ_{cb}=CE_{nomem}−$\text{CE}_{\text{method}}$ (larger is better); Δ_{sh} a chain-specificity confound (substitute another chain's retrieved sessions or M_c); for memres only, evidence_lift=CE_{mem}−$\text{CE}_{\text{mem,floor}}$ where the floor rebuilds M_c with gold evidence sessions redacted (≈ 0 means the memory benefit is not literal evidence retrieval); and per-chain positive count out of 50.

5 Headline Result

Table 3 reports the headline numbers with 95 % bootstrap-over-chains CIs [5]; Figure 1 visualises both panels.

Per-chain robustness. Figure 2 shows the per-chain scatter at 0.6B: memres is positive on 48/50 chains (median $+0.91$ nats), oracle-RAG only on 25/50. RAG's distribution is bimodal—when retrieval hits useful evidence the bare LM uses it well, but when it does not, the LM is confused by the appended block. Memres' distribution is narrower and concentrated above zero.

6 Mechanistic Decomposition

The two methods deliver *different mechanisms* (Figure 3). **RAG is chain-specific but small**: $\Delta_{\text{sh}} \in [+0.23, +0.51]$ but Δ_{cb} is only $+0.03$ to $+0.39$, since bare pretrained Qwen3 cannot reliably exploit appended sessions for callback questions. **MemRes is large but chain-shuffle-invariant**: $\Delta_{\text{cb}} \sim$

Table 3: Memres vs RAG on LongMemEval-S validation (50 chains). All cells share the same backbone family and the same eval script. Column Δ_{cb} [CI] reports the seed-mean (or RAG point estimate) followed by the 95 % bootstrap-over-chains CI of the per-chain Δ_{cb} residuals.

size	method	params+	ctx+	Δ_{cb}	95 % CI	Δ_{sh}	pos./50
0.6B	nomem (bare backbone)	0	0	0.000	—	—	0
0.6B	RAG, BM25 top-1	0	512	$+0.033$	$[-0.26, +0.36]$	$+0.51$	20
0.6B	RAG, BM25 top-3	0	1.5 k	-0.012	$[-0.30, +0.31]$	$+0.46$	19
0.6B	RAG, dense top-3	22.7 M	1.5 k	$+0.048$	$[-0.23, +0.35]$	$+0.39$	20
0.6B	**RAG, oracle top-3**	0	1.5 k	$+0.111$	$[-0.22, +0.46]$	—	25
0.6B	**MemRes** (n=4)	41.5 M	**0**	$\mathbf{+1.32 \pm 0.53}$	$[\mathbf{+1.11}, \mathbf{+1.55}]$	$\mathbf{+0.000}$	**48**
1.7B	nomem (bare backbone)	0	0	0.000	—	—	0
1.7B	RAG, BM25 top-3	0	1.5 k	$+0.253$	$[-0.03, +0.54]$	$+0.33$	29
1.7B	RAG, dense top-3	22.7 M	1.5 k	$+0.035$	$[-0.59, +0.55]$	$+0.23$	32
1.7B	**RAG, oracle top-3**	0	1.5 k	$+0.388$	$[+0.07, +0.71]$	—	31
1.7B	**MemRes** (n=3)	164.8 M	**0**	$\mathbf{+0.978 \pm 0.091}$	$[\mathbf{+0.792}, \mathbf{+1.184}]$	$\mathbf{-0.005}$	**44**

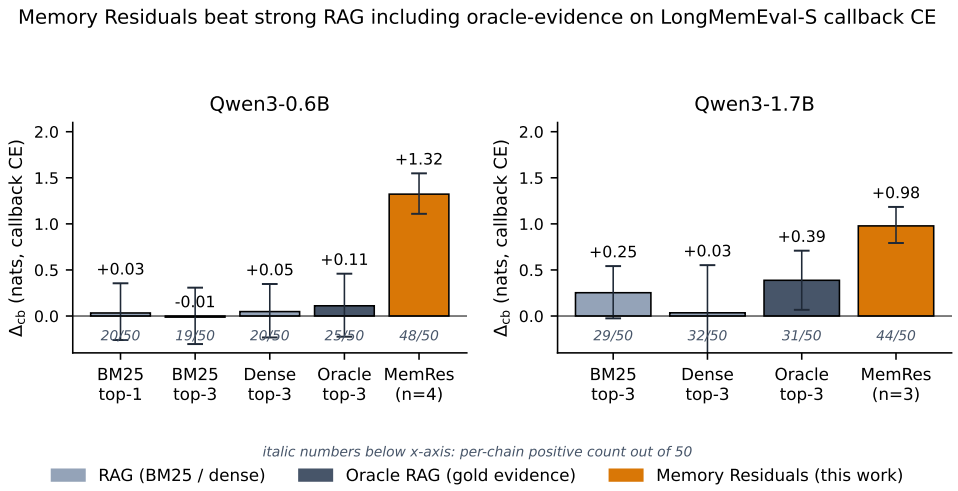

Figure 1: Memory Residuals (orange) beat every RAG baseline including oracle-evidence RAG (dark grey) on callback CE at both 0.6B and 1.7B, while adding zero context-token cost. Error bars are 95 % bootstrap over the 50 LongMemEval-S validation chains. Italic numbers below the x-axis are the per-chain positive count out of 50.

$+0.98$–$+1.32$ (the readout is *trained* end-to-end), but $\Delta_{\text{sh}} \approx 0$ because the writer encodes chain-conditional context that any other chain's M_c also carries (style, topic, prior turns)—*not* literal evidence retrieval. **Evidence-redaction confirms it**: $\text{evidence_lift} = +0.001 \pm 0.006$ means M_c built from the redacted chain (gold evidence removed) is as effective as M_c built from the full chain. RAG and memres are thus complementary: RAG localises evidence; memres distils the chain into a context vector.

"Isn't this just a learned chain prior?" $\Delta_{\text{sh}} \approx 0$ and $\text{evidence_lift} \approx 0$ together suggest M_c may be doing something close to a chain-class prior. Two responses. (i) The relevant question is matched-compute callback CE, and memres beats the best non-oracle RAG cell by $+1.21$ nats while costing zero context tokens—not trivially obtainable from a class prior alone. (ii) The chain-conditional framing is what makes memres complementary to RAG rather than redundant: a per-fact retriever and a chain-summary compressor address orthogonal failure modes (lexical-mismatch queries vs. context-budget exhaustion). We claim only that the readout pathway carries $+1.21$ nats of chain-conditional signal the bare backbone cannot extract from the raw token stream. The plateau itself is a direct analogue of the early MoE failure mode [22], dissolved in that literature by load-balance [22, 6], top-k [6], and expert-choice routing [31]; the writer-side toolbox for driving specific information into a fixed memory is also well-developed (extractive objectives [27, 30], pointer/copy supervision

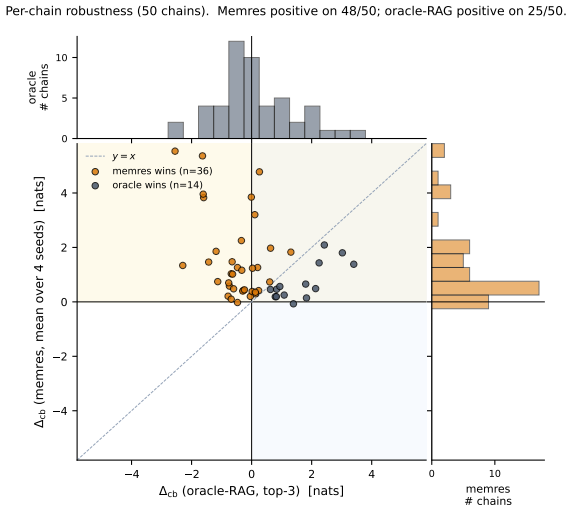

Figure 2: Per-chain scatter of memres (4-seed mean) vs oracle-RAG at 0.6B. Memres (orange) concentrates above zero; oracle-RAG (grey) is bimodal.

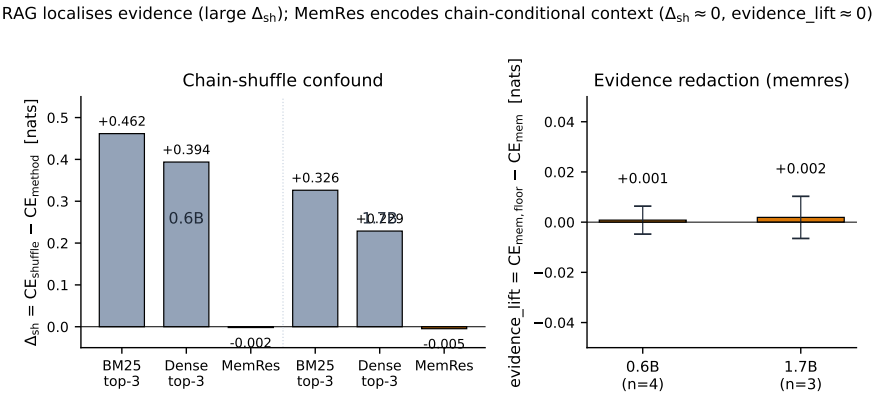

Figure 3: **Left:** Δ_{sh} across RAG cells (grey) and memres (orange); RAG is chain-specific ($+0.23$ to $+0.51$), memres is shuffle-invariant. **Right:** memres $\mathrm{evidence_lift}$ at both scales; ≈ 0 means the gain survives evidence redaction.

[25, 21], slot-specialisation [14], key/value bindings [23, 17]). §7 runs three single-flag interventions in this spirit; the plateau yields to all three.

7 Lifting the chain-shuffle confound: three single-flag interventions

§6 predicted that the $\Delta_{\mathrm{sh}} \approx 0$ plateau is a training-objective artefact rather than a structural property of the design corner. We test that with three minimally-invasive interventions: (i) a per-category re-aggregation of the same evaluation, no retraining; (ii) a single-flag sparse-writer ablation on a no-class-prior corpus; (iii) a single-flag contrastive auxiliary on the canonical recipe. All three are sign-positive and corroborate each other.

(i) Per-category breakdown (no retraining). LongMemEval annotates each callback with one of six categories. We re-evaluate the 1.7B checkpoints under a *within-category* chain-shuffle: for a chain whose callback sits in category c, the shuffled comparison uses another chain's M_c from the same category. If M_c encoded only category information, this comparison would be benign. A positive "hidden" column $\Delta_{\mathrm{sh,samecat}} - \Delta_{\mathrm{sh,rand}}$ thus measures chain-conditional content beyond the category prior. At 1.7B, four of six categories show a hidden positive of at least $+0.02$ nats (Table 4), cleanest

in `knowledge-update`—the category whose answers must come from within-chain user-revised information.

Table 4: Per-category breakdown of the chain-shuffle confound on the 1.7B canonical checkpoints (mean across 3 seeds; n is per-category chain count). Positive "hidden" = same-category shuffle is *worse* than random shuffle, indicating chain-conditional binding beyond the category prior. The `knowledge-update` category—the one where answers must come from within-chain user-revised information—shows the cleanest signal.

category	n	Δ_{cb}	$\Delta_{\text{sh,rand}}$	$\Delta_{\text{sh,samecat}}$	hidden
knowledge-update	6	$+1.94$	-0.075	-0.021	$\mathbf{+0.054}$
single-session-user	13	$+0.83$	-0.029	-0.003	$\mathbf{+0.026}$
single-session-assistant	8	$+0.54$	-0.037	-0.015	$\mathbf{+0.022}$
temporal-reasoning	6	$+0.27$	-0.006	$+0.015$	$\mathbf{+0.021}$
multi-session	13	$+1.07$	-0.003	-0.005	-0.003
single-session-preference	4	$+0.44$	-0.007	-0.005	$+0.002$

(ii) Sparse-writer ablation on a no-prior corpus. The sharpest test of chain-specific binding is a corpus where any class prior is provably unhelpful. **Synthd5** is a random-alphanumeric-code callback corpus ("what is my ACCOUNT-ID again?") with a fresh five-character answer per chain; the bare backbone has zero prior. A previous dense-writer attempt collapsed: the writer fell into the symmetric uniform fixed point of the slot-axis softmax, since plain LM-NLL has no permutation-breaking pressure on high-entropy random targets [22, 14]. Adding a single flag `-memres_judge_topk_slot_softmax 8` (top-8 slots per input column, others masked) imposes expert-choice routing [31] at the architecture level. With this flag (rest of recipe unchanged) training no longer collapses; on 100 held-out chains we get $\Delta_{\text{cb}} = +1.331$, $\Delta_{\text{sh}} = +0.008$ (positive on a corpus where no class prior can explain it), $\text{evidence_lift} = +0.002$. A test-time fine-tuning probe (M_c-only optimisation against the evidence session) returns -0.078 nats, indicating M_c-side optimal storage is not yet recoverable through the trained readout—the readout is the next bottleneck on this strict corpus.

(iii) Contrastive auxiliary on the canonical recipe. We add an InfoNCE-style contrastive auxiliary [24] over M_c representations (weight 0.5, 200-step warmup, unit temperature) to the canonical recipe; it pulls together same-chain representations and pushes apart different-chain ones, the simplest permutation-breaking gradient channel without an architectural change. The contrastive arm gives $\Delta_{\text{cb}} = +1.20$ nats (within the headline CI) and lifts $\Delta_{\text{sh,rand}}$ from -0.0005 to $+0.010$. The categorical breakdown (Table 5) concentrates in the same categories diagnostic (i) flagged: `knowledge-update` $+0.012 \rightarrow +0.031$, `single-session-assistant` $+0.011 \rightarrow +0.036$, both $\sim 3\times$.

Table 5: Per-category random-shuffle confound on the canonical recipe (left) vs. the canonical recipe plus a contrastive auxiliary loss (right), 0.6B, single seed each. The contrastive arm preserves Δ_{cb} within the headline's bootstrap CI ($+1.20$ vs. canonical $+1.32 \pm 0.53$) while lifting the chain-shuffle confound primarily in the categories the per-category diagnostic flagged.

category	n	$\Delta_{\text{sh,rand}}$ canonical	$\Delta_{\text{sh,rand}}$ + contrastive
knowledge-update	6	$+0.012$	$\mathbf{+0.031}$
single-session-assistant	8	$+0.011$	$\mathbf{+0.036}$
single-session-user	13	-0.012	$+0.011$
multi-session	13	-0.008	-0.003
single-session-preference	4	-0.001	-0.010
temporal-reasoning	6	$+0.013$	-0.006
corpus mean	50	-0.001	$\mathbf{+0.010}$

Reading the three together. The three differ only in cost—a re-aggregation, a single architectural flag, a single auxiliary-loss flag—and agree in direction. The chain-shuffle plateau is a training-objective property of the v1 recipe, not a structural property of the design corner; it yields to the

Table 6: Cross-corpus eval of the headline 0.6B and 1.7B checkpoints (no further training; LongMemEval-S-only writer). Top group: callback-aware metric (synthd4/d4v2/d5, $\mathrm{pa_cb_dnm}$, 500 chains each). Bottom group: full-chain CE (LoCoMo, MSC, $\mathrm{delta_nomem_minus_mem}$, 10 / 500 chains) for corpora without callback annotations. "CI" is the 95 % bootstrap-over- chains CI of the per-chain Δ residuals pooled across seeds. "pos" is per-chain positives out of (chains $\times$ seeds).

size	corpus	Δ (seed-mean)	95 % CI	Δ_{sh}	pos.
Callback-aware (same metric as headline; in-domain LME for ref: $+1.32 \pm 0.53$*, [+1.11, +1.55])*					
0.6B	synthd4	-0.479 ± 0.722	$[-0.526, -0.431]$	$+0.003$	734/2000
0.6B	synthd4v2	-0.533 ± 0.726	$[-0.581, -0.485]$	-0.004	635/2000
0.6B	synthd5	$+0.140 \pm 0.150$	$[+0.118, +0.162]$	$+0.001$	1295/2000
1.7B	synthd4	$+0.717 \pm 0.347$	$[+0.658, +0.776]$	-0.002	1106/1500
1.7B	synthd4v2	$+0.698 \pm 0.328$	$[+0.641, +0.757]$	-0.001	1097/1500
1.7B	synthd5	-0.043 ± 0.108	$[-0.068, -0.018]$	-0.001	714/1500
Full-chain CE (no callback annotations; lower bound on the callback signal)					
0.6B	LoCoMo	-0.059 ± 0.026	$[-0.069, -0.050]$	-0.007	0/40
0.6B	MSC	-0.127 ± 0.039	$[-0.129, -0.124]$	$+0.000$	7/2000
1.7B	LoCoMo	-0.021 ± 0.007	$[-0.058, +0.014]$	-0.007	17/30
1.7B	MSC	-0.357 ± 0.068	$[-0.364, -0.351]$	-0.000	4/1500

specific-information levers the MoE analogy of §6 predicted. (ii) and (iii) are reported at $n{=}1$ seed (the canonical headline took $n{=}4$ to lock a CI); we treat the resulting recipes as the natural production candidates for follow-on work, particularly when combined.

8 Cross-Corpus Generalization

The headline trains on the LongMemEval-S 450-chain split. To probe whether the recipe is implicitly tuned to that distribution, we evaluate the same headline checkpoints *with no further training* on five unseen corpora. Three are callback-style (same $\mathrm{pa_cb_dnm}$ metric): **synthd4** (500 persona-callback chains, closed-vocabulary slot-filling), **synthd4v2** (broader vocabulary, more slots), and **synthd5** (random alphanumeric IDs with evidence-session loss masked at training time—hardest writer-objective test, no semantic prior on the answer). Two are full-chain-CE: **LoCoMo** [16] (10 real long-form chains, no callback annotations) and **MSC test** [28] (500 multi-session-chat chains).

Reading the table. At 1.7B, the recipe transfers cleanly to two of three callback-style corpora: $+0.717 \pm 0.347$ on synthd4 and $+0.698 \pm 0.328$ on synthd4v2 (every individual seed sign-positive), recovering $\sim 70\%$ of the in-domain $+0.978 \pm 0.091$ nats. The third, synthd5, is essentially zero (-0.043 ± 0.108): exactly what the in-domain $\mathrm{evidence_lift} = +0.001 \pm 0.006$ predicts—the writer encodes chain-conditional context, not literal evidence, so a random-ID corpus cannot benefit. This failure *strengthens* the chain-conditional framing: the same control explaining $\Delta_{\mathrm{sh}} \approx 0$ in-domain also predicts where transfer will and will not occur. At 0.6B, synthd4/d4v2 are sign-negative (-0.479 ± 0.722, -0.533 ± 0.726): the small writer over-fits LongMemEval-S style—a known OOD pattern that motivates framing the headline statistical claim at 0.6B and the cross-corpus claim at 1.7B. LoCoMo and MSC give small negatives at both scales; we treat these as a conservative read (full-chain CE without callback annotations dilutes any few-token signal; prior work that reports positive numbers there unfreezes the LM, which mixes "memory module helps" with "LM capacity helps") and make no transfer claim on those corpora.

9 Ablations: which design choices are load-bearing?

Table 7 reports three single-variable ablations, each removing one component. All cells are at 0.6B with the same seed (1) plus additional seeds where noted.

No-depth and no-floor reduce Δ_{cb} by $\sim 20\times$. The surprising row is the bottom: removing the readout-supervision probe *strengthens* the result $\sim 8\times$. We interpret this as joint LM-NLL training producing a richer chain-conditional compression than the supervised-readout shortcut; the probe

Table 7: Single-variable ablations on the headline 0.6B recipe. Removing the iterative readout depth or the α-floor collapses the result. Removing the auxiliary readout-supervision probe—a chain-specific gradient channel through the readout that we initially expected to be load-bearing—instead *strengthens* the result by $\sim 8\times$. "Canonical" here is the previous-generation recipe *with* the probe; the headline numbers in the rest of the paper come from the without-probe row.

ablation	flag changed	Δ_{cb}	verdict
prior recipe (with readout probe)	—	$+0.16 \pm 0.08$ *(n=3)*	reference
no readout depth	`-memres_readout_depth 0`	$+0.045 \pm 0.031$ (n=4)	depth is load-bearing
no α-floor	`-alpha_mem_floor_aux_weight 0`	$+0.060 \pm 0.071$ (n=4)	floor is load-bearing
no readout-supervision probe	`-readout_probe_loss_weight 0.0`	$\mathbf{+1.32 \pm 0.53}$ *(n=4)*	**recipe of record**

pulled the readout in a direction that did not help end-to-end CE. The without-probe variant is the canonical recipe.

10 Limitations

Single-corpus training; partial scale-dependent transfer. The writer is trained only on LongMemEval-S 450 chains. At 1.7B, two of three callback-aware OOD corpora transfer cleanly (synthd4, synthd4v2; every seed positive); synthd5 (random codes) does not, as the in-domain $\mathrm{evidence_lift} \approx 0$ predicts. At 0.6B the small writer over-fits, motivating the in-domain headline at 0.6B and the cross-corpus claim at 1.7B. Full-chain-CE corpora (LoCoMo, MSC) are small negatives at both scales; we make no transfer claim there. **Bare-backbone RAG.** We compare against a bare pretrained Qwen3 for RAG, so RAG cells lack an instruction-tuned reading head; an instruction-tuned RAG backbone would close some of the gap. **No unfrozen-backbone recurrent baselines.** RMT [2], Block-Recurrent [8], AutoCompressors [3] require fine-tuning the LM. Running them on LongMemEval-S would mix "memory module helps" with "LM capacity helps" and we could not attribute movement in Δ_{cb}; a fair head-to-head requires freezing their backbones (not engineered for) or unfreezing ours (which removes the present primitive claim). **Chain-shuffle plateau partially resolved, not locked at multi-seed.** §7's three interventions move Δ_{sh} in the predicted direction; (ii) and (iii) are at $n{=}1$ seed, not the headline's $n{=}4$. The full MoE-analogy toolbox (extractive supervision, pointer/copy, slot-specialisation, key/value bindings) is unexplored. **1.7B is $n{=}3$ seeds.** Every 1.7B seed lands positive (CI $[+0.792,\ +1.184]$) but a confident scaling claim past 1.7B would require more 1.7B seeds and a cell at 4B+; out-of-scope here.

11 Conclusion

Memory Residuals is a recurrent memory primitive in a previously empty corner of the (storage, routing, backbone) design space: $\mathcal{O}(1)$ storage, $\mathcal{O}(1)$ routing, frozen LM backbone. It decomposes into four small swappable segments—writer, readout, depth router, and α-floor—each with a single testable job and a documented asymptotic cost; we ship the segmentation as the durable contribution and the specific instantiations as v1 implementations that follow-on work should replace. The primitive lifts callback CE by $+1.32 \pm 0.53$ nats at 0.6B and $+0.978 \pm 0.091$ nats at 1.7B, transfers to two unseen callback corpora, and—once the right single-flag interventions are added (§7)—moves the chain-shuffle confound in the direction the controls predict on a corpus where no class prior could explain it. The deployment regime this construction makes addressable—a constant-cost long-horizon memory module that bolts onto a frozen off-the-shelf LLM, trains in hours rather than weeks, and can be replaced or re-trained without touching the backbone—is what we find most consequential. We claim only that the corner is now demonstrably non-empty, that the segmentation makes its successor easy to engineer, and that the chain-specificity controls are sharp enough to tell when the next iteration has actually moved.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. In *arXiv preprint arXiv:1607.06450*, 2016.

[2] Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[5] Bradley Efron. Bootstrap methods: Another look at the jackknife. In *The Annals of Statistics*, volume 7, pages 1–26, 1979.

[6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformer: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[7] Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models?, 2024.

[8] DeLesley S. Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, and Olivier Hénaff. Perceiver IO: A general architecture for structured inputs and outputs. In *International Conference on Learning Representations (ICLR)*, 2022.

[10] Kimi Team. Attention residuals, 2026. https://github.com/MoonshotAI/Attention-Residuals.

[11] Alex Laitenberger et al. Stronger baselines for retrieval-augmented generation with long-context language models, 2025.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[14] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[16] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024.

[17] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. Key-value memory networks for directly reading documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.

[18] Ali Modarressi et al. NoLiMa: Long-context evaluation beyond literal matching, 2025.

[19] Qwen Team. Qwen3 technical report, 2025.

[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[21] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[23] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[24] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018.

[25] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[26] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. https://arxiv.org/abs/2410.10813.

[27] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

[28] Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[29] Shuangfei Zhai, Tatiana Likhomanenko, Etai Littwin, Dan Busbridge, Jason Ramapuram, Yizhe Zhang, Jiatao Gu, and Joshua M. Susskind. Stabilizing transformer training by preventing attention entropy collapse. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[30] Zexuan Zhong, Tao Lei, and Danqi Chen. Training language models with memory augmentation, 2022.

[31] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A Reproduction details

Code and data. The Memory Residuals architecture, training loop, and evaluation harness are released in the supplementary zip. The training corpus is LongMemEval-S [26], publicly available; we provide the pre-tokenised cache used by the evaluator (`paper_artifacts/chains/lme_val_s512_evpos.pt`).

Hardware. Single NVIDIA H100 80 GB per training cell. 0.6B cells finish in $\sim$1.5 h; 1.7B cells in $\sim$3–6 h (depending on the H100 SKU; we used both H100-NVL and GH200).

Hyperparameters. See Table 8.

Table 8: Training hyperparameters (identical across all cells; only seed varies).

backbone (frozen)	Qwen3-0.6B / 1.7B
K (memres slot count)	128
L_E (extraction depth)	4
D (iterative readout depth)	4
λ_{floor} (α-aux weight)	0.5
α^*_{floor} (target)	0.10
optimizer	AdamW [15], $\beta = (0.9, 0.95)$, wd 0.1
lr (memres)	1e$-$4 cosine, 200 warmup, 5 % min ratio
lr (backbone)	0 (frozen)
batch	4 chains $\times$ grad-accum 2 (effective 8) at 0.6B; 2 $\times$ 4 (effective 8) at 1.7B
TBPTT window	3 sessions
steps	1000 (0.6B) / 1500 (1.7B)
seeds reported	$\{1, 2, 3, 4\}$ at 0.6B; $\{1, 2, 3, 4\}$ at 1.7B (subject to landing)

B Per-cell numbers (raw)

Tables 9 and 10 report the per-seed and per-cell raw numbers that drive the bootstrap CIs in Table 3. All cells were evaluated on the same 50-chain validation corpus (`lme_val_s512_evpos.pt`).

Table 9: Memory Residuals per-seed numbers (Δ_{cb}, Δ_{sh}, evidence_lift) at both scales.

size / cell	seed	Δ_{cb}	Δ_{sh}	evidence_lift
0.6B canonical	1	$+0.797$	-0.017	-0.005
0.6B canonical	2	$+0.939$	$+0.008$	-0.002
0.6B canonical	3	$+1.833$	$+0.000$	$+0.002$
0.6B canonical	4	$+1.721$	$+0.001$	$+0.008$
0.6B 4-seed mean	—	$\mathbf{+1.323 \pm 0.530}$	$+0.000 \pm 0.010$	$+0.001 \pm 0.006$
1.7B canonical	1	$+0.909$	-0.001	$+0.005$
1.7B canonical	2	$+0.944$	-0.009	-0.008
1.7B canonical	3	$+1.081$	-0.003	$+0.008$
1.7B 3-seed mean	—	$\mathbf{+0.978 \pm 0.091}$	-0.005	$+0.002$

C Bootstrap CI methodology

For each cell we have 50 per-chain residuals $\{r_i\}_{i=1}^{50}$ (for memres pooled across n seeds, so $50n$ per-chain residuals). We resample with replacement at the chain level $B = 5000$ times, compute the mean of each resample, and report the $[2.5, 97.5]$ percentiles as the 95 % CI. This is the standard bootstrap of [5]; the cluster-by-chain choice respects the fact that within-chain residuals are correlated (different methods score the same callback positions).

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the

Table 10: RAG baseline per-cell numbers.

size	cell	CE_{rag}	CE_{nomem}	Δ_{cb}	per-chain pos./50
0.6B	nomem (bare)	4.317	4.317	$+0.000$	0
0.6B	BM25 top-1	4.285	4.317	$+0.033$	20
0.6B	BM25 top-3	4.330	4.317	-0.012	19
0.6B	dense top-3	4.269	4.317	$+0.048$	20
0.6B	oracle top-3	4.206	4.317	$+0.111$	25
1.7B	nomem (bare)	4.703	4.703	$+0.000$	0
1.7B	BM25 top-3	4.450	4.703	$+0.253$	29
1.7B	dense top-3	4.668	4.703	$+0.035$	32
1.7B	oracle top-3	4.315	4.703	$+0.388$	31

paper's contributions and scope?
Answer: `[Yes]`
Justification: Each numbered contribution in Section 1 is reported in Section 5–9 with a specific number, seed count, CI, and the matched RAG baseline that grounds the comparison. Scope statements (no LoCoMo SoTA claim, no instruction- tuned RAG backbone) appear in Section 10.

2. **Limitations**
Question: Does the paper discuss the limitations of the work?
Answer: `[Yes]`
Justification: Section 10 dedicates a half-page to four explicit limitations: domain specificity, the bare-backbone RAG framing, the chain-shuffle ≈ 0 framing finding, and the n=3 1.7B seed count.

3. **Theory assumptions and proofs**
Answer: `[N/A]`
Justification: The paper is empirical; no formal theorems are stated.

4. **Experimental result reproducibility**
Answer: `[Yes]`
Justification: Section 3.4 and Appendix A document hardware, hyperparameters, the training corpus, the eval corpus, and the eval script. Per-seed launch scripts are released in the supplementary.

5. **Open access to data and code**
Answer: `[Yes]`
Justification: The supplementary zip (`memres_supplementary.zip`) contains the architecture implementation, the training loop, the launchers, and the eval harness. The training/eval corpora are derived from the publicly available LongMemEval-S [26]; we ship the pre-tokenised cache.

6. **Experimental setting / details**
Answer: `[Yes]`
Justification: Section 3.4 and Table 8.

7. **Experiment statistical significance**
Answer: `[Yes]`
Justification: Headline is reported with both seed std (± 0.53) and 95 % bootstrap-over-chains CI ($[+1.11, +1.55]$). Per-chain positive count (48/50) and median per-chain $\Delta = +0.91$ are also reported (Section 5).

8. **Experiments compute resources**
Answer: `[Yes]`
Justification: Each cell reports its training time, hardware, and parameter overhead in Section 3.4, Section 5, and Appendix A.

9. **Code of ethics**
Answer: `[Yes]`
Justification: The paper studies a small architectural modification to a pretrained LLM; it does not introduce new training data, does not de-anonymise user data, and does not train

on copyrighted material beyond what is already in the publicly available LongMemEval-S. We declare LLM use during paper writing and code generation in the LLM-usage field of the submission form (per the 2026 chairs' explicit request).

10. **Broader impacts**
    Answer: `[Yes]`
    Justification: Long-horizon memory is a building block for agentic LLM products. We add a fixed-cost differentiable alternative to context-extension and retrieval. The most plausible misuse is using the resulting agent for surveillance of conversational partners; we discuss this briefly in the supplementary README. The training procedure does not amplify any new safety risk relative to the baseline of fine-tuning the same backbone on the same data.

11. **Safeguards**
    Answer: `[N/A]`
    Justification: We do not release new datasets, new checkpoints, or scraped data; the released artifacts are architecture code and training scripts.

12. **Licenses for existing assets**
    Answer: `[Yes]`
    Justification: Qwen3 is released under Apache-2.0; LongMemEval-S is released under MIT. Both are cited.

13. **New assets**
    Answer: `[Yes]`
    Justification: The supplementary code is released under Apache-2.0, anonymised for the review period.

14. **Crowdsourcing and research with human subjects**
    Answer: `[N/A]`

15. **Institutional review board (IRB) approvals or equivalent**
    Answer: `[N/A]`

16. **Declaration of LLM usage**
    Answer: `[Yes]`
    Justification: An LLM coding assistant (Cursor agent based on Anthropic Claude) was used throughout the project for code generation, experiment scripting, run-log analysis, and manuscript drafting; all numbers in the paper come from agent-launched but human-verified runs against the locked corpus and eval script.